

Two Level Partitioning

L: Numero Di elem. in ogni Blocco

1. Divido in Blocchi la lista più lunga

2. Costruisco A' che contiene i Primi Elem. di ogni Blocco

3. Dato B in Blocchi (1 v Blocco A) $\rightarrow$ ci sono Tutti gli elm. di $B \in 1^{\circ}$ Elm del Blocco ... ultimo elm del Blocco

Possò anche avere insieme Vuoti:

4. Faccio un'intersezione fra i blocchi A_i e B_i

Neutral Partitioning

chiamo B la lista più corta.

Scego il suo elemento mediano $\rightarrow$ Se $E \in A$: appartenenza all'intersezione
 $\downarrow$
 lo faccio con la Binary Search

! ogni pivot elimina metà della lista corta

Ricorvo su Ponte sinistra di A_L e B_L e Ponte dx A_R e B_R

$\downarrow$ $\downarrow$

Elementi < di p Elementi > di P

Minimal Perfect Hash

1. Ordinare le stringhe ed associare il rank $(0, n-1)$

2. Calcolare h_1 ed h_2 per ogni stringa

3. Costruire un grafo dove si collega ogni modo di h_1 con h_2 etichettato con il rank della chiave. $\rightarrow$ ogni chiave diventa un arco del grafico

- Se il grafo non ha cicli = Top

- Se il grafo ha cicli bisognerebbe ricompilarlo ma in un esercizio costruirlo. Se non è possibile basta segnalare

$$4. h(s) = (g(h_1(s)) + g(h_2(s))) \bmod \underset{\substack{\uparrow \\ \text{numero delle stringhe}}}{n}$$

Fondamentalmente vale $h(s) = r(s) \Rightarrow g(h_1(s)) + g(h_2(s)) \equiv r(s)$ questo significa che ogni arco produce come eq $\overset{u}{g(h_1(s))} + \overset{v}{g(h_2(s))} \equiv r \pmod{n}$

Se ho un arco (u, v) con rank r allora $g(u) + g(v) \equiv r \pmod{n}$

$$\text{Conoscendo } g(u) \rightarrow g(v) \equiv r - g(u) \pmod{n}$$

$$g(u) \equiv r - g(v) \pmod{n}$$

Al Termine $\forall$ chiave $h(s) = (g(h_1(s)) + g(h_2(s))) \bmod n$ e controllo che venga il rank corretto

Canonical Huffman code

1. Ordino i simboli per Probabilità crescente
2. Costruisco l'albero di Huffman andando a fondere sempre i rami minimi $\rightarrow$ Huffman canonico da cui si ricavano le lunghezze dei codici
3. Costruisco symb symb[E] = tutti i simboli con lunghezza E
4. costruisco num num[E] = # simboli codificati con tale lunghezza
5. $FC[max] = 0$ $FC[i] = \frac{FC[i+1] + num[i+1]}{2}$
6. Per ogni livello E partendo da FC[E] e assegno i codici consecutivi a symb[E] scrivendoli su E bit

Decodifica Canonical Huffman

- Leggo un bit $vel_{10} < FC[E]$ leggo un altro bit
- $vel_{10} \geq FC[E]$ mi fermo $V - FC[E] = offset$
- symb[offset]

Arithmetic Encoding

1. Costruire la Probabilità cumulativa $F(x) = \sum_{y \leq x} P(y)$
2. $[L_i, R_i + S_i)$ $R_i = R_{i-1} + S_{i-1} \cdot F(S[i])$
- $S_i = S_{i-1} \cdot P(S[i])$
- $R_i + S_i = R_{i-1} + S_{i-1} \cdot F(S[i]) + S_{i-1} \cdot P(S[i])$
- $= R_{i-1} + S_{i-1} \cdot (F(S[i]) + P(S[i]))$

Alternativamente: $[C_{low}, C_{high})$ $L' = L + Range \cdot C_{low}$ $Range = U - L$ U, B : la lunghezza finale dipende dalle Prob. dei simboli

$U' = L + Range \cdot C_{high}$

L377

Montando • W = Testo già visto • B = Testo ancora da comprimere

Ad ogni Passo: 1) Guardare il Prefisso del buffer B

2) Cerco la sua occorrenza più lunga in W

3) Produrre una Trippla (d, l, c) • d: distanza dall'occorrenza

• l: lunghezza del match

• c: Primo carattere dopo match

4) Avanzare di L+1 caratteri

Decodifica

Per ogni Trippla (d, l, c) andare indietro di d Posizioni, copiare l caratteri

E aggiungere c.

No match: (0, 0, c)

In caso di match scegliere il vicino (occhio ad eventuali indicatori)

L355

1. Cercare il match più lungo del Prefisso del buffer dentro la finestra

2. Se il match è abbastanza lungo emettere (d, l) altrimenti emettere il carattere locale

3. Avanza di l o di 1

Decodifica: se leggo un letterale lo riporto in output altrimenti vedo dietro di d posizioni e copio l caratteri

Unary Code* Unary(x) = $1^x 0$

Gamma Code 1. $L = \lfloor \log_2(x) \rfloor$ 2. Scrivere x in binario 3. $\text{Gamma}(x) = \text{Unary}(L) + \text{binario } x \text{ senza primo 1}$

Decodifica 1. Leggi unary $\rightarrow$ conta gli unary prima dello 0
2. Leggi le bit
3. $x = 2^L + \text{suffix}$

Delta Code 1. $L = \lfloor \log_2(x) \rfloor + 1$
2. Codifica L con gamma Delta(x) = gamma(L) + suffix(x)
3. Aggiungi binario di x senza primo 1

Decodifica

1. Decodifica un gamma e ottengo L
2. Leggi L-1 bit
3. Metti avanti 1 $\rightarrow x = 1$ concatenato ai L-1 bit letti

Rice Code

1. $q = \left\lfloor \frac{x}{2^k} \right\rfloor$ $r = x \bmod 2^k$
 2. Codifica unary q
 3. Codifica r su k bit
- $\text{Rice}_k(x) = \text{Unary}(q) \parallel \text{bin}_k(r)$
 $\uparrow$
Parametro

Decodifica 1. Decodifica unary e ottengo q
2. Leggi k bit e ottengo r
3. $x = q2^k + r$

S, C Dense in fatto?

Variable Byte

1. Scrivere x in binario
 2. Spezzare in gruppi da 7 bit
 3. Completare il gruppo più a sinistra con 0 se necessario
 4. Aggiungi in testa ai gruppi un bit di controllo
- 0 se ci sono altri bit • 1 se è l'ultimo byte

Decodifica Leggi 8 bit; Togli 1 bit di controllo; concatena i 7 bit; in base al bit di controllo ordina o meno avanti

Per Delta

Vengono dati due Parametri: base, b $\rightarrow$ fissa codifica [base, base + $2^b - 2$]

Se $x \in \text{Range} \rightarrow$ Scrivere $x - \text{base}$ in b bit

$x \notin \text{Range} \rightarrow$ Scrivere Escape $2^b - 1$ in b bit e metto x nella lista Eccezioni

Decodifica

1. Leggi un campo da b bit
2. Se non è escape Restituire base + valore

Se escape Prendere prossimo valore dalla lista Eccezioni

Elas Fano

1. $l = \lceil \log_2 \frac{u}{m} \rceil$ max Num
Numeri

2. $\forall x_i: \text{low}_i = x_i \bmod 2^l, \text{high}_i = \lfloor \frac{x_i}{2^l} \rfloor$

3. Scrivere tutti i low in array L ciascuno su l bit

4. Codifica gli high in un bitstream $\rightarrow H_S$ contiene Tutti i quanti sono i Numeri la cui Parte alta e 1 chiuso da 0

Decodifica / Accena

- Prendi l bit del low di x_i da L
- $P = \text{select}_1(M, i)$
- $\text{high}_i = P - i$
- $x_i = \text{high}_i \cdot 2^l + \text{low}_i$

Next Gg

- $x_h = \lfloor \frac{x}{2^l} \rfloor, x_l = x \bmod 2^l$
- $P = \text{select}_0(M, x_h) + 1 \quad (x_h = 0 \rightarrow P = 1)$ Bucket $H[P, q]$
- $q = \text{select}_0(M, x_h + 1)$
- Se x_h vuoto $\rightarrow$ Risultato sta nel 1° Bucket Non Vuoto successivo
- In Pratica si cerca il primo uno a destra di P
- Se x_h pieno $\rightarrow i_{\text{start}} = P - x_h, i_{\text{end}} = q - x_h - 1$

il Num sta in $[i_{\text{start}}, i_{\text{end}}] \rightarrow$ 0 basso con BS o faccio Accena

Interpolative Coding

$l = 1, r = m, \text{low} = s'_r, \text{hi} = s'_m$
Caso base: $l = r: \text{Binary Code}(S[l], \text{low}, \text{hi})$
Oth: $m = \lfloor \frac{l+r}{2} \rfloor$ e codifico s'_m dove $\text{low}' = \text{low} + m - l$
 $\text{high}' = \text{high} + m - r$
 $\text{Binary Code}(S[m], \text{low}', \text{high}')$

Defo ricorro a sinistra e aggiorno $\text{high} = S[m] - 1, \text{low}$ resta uguale
Destra $\text{low} = S[m] + 1, \text{high}$ uguale

Binary Code($S[l], \text{low}, \text{hi}$)
- $\text{mbit} = \lceil \log(\text{hi} - \text{low} + 1) \rceil$
- codifico $S[l] - \text{low}$ su mbit

Interpolative($s', l, m-1, \text{low}, \text{high}$)
Interpolative($s', m+1, r, s'[m]+1, \text{hi}$)

Rank

$$i = 1 + \left\lfloor \frac{x-1}{2} \right\rfloor$$

$$j = 1 + \left\lfloor \frac{(x-1) \bmod 2}{2} \right\rfloor$$

$$o = 1 + ((x-1) \bmod 2)$$

$$\text{Rank}_s(x) = r_i + r_{i,j} + R[B_{i,j}, o]$$

N.B.: Struttura occupa $O(m)$

Succinct Encoding Binary Tree

"Modifica l'albero andando a mettere delle Dummy leaves dove necessario. Ogni nodo deve avere 2 figli."

Visita l'albero per livelli e scrivi il Bitvector: 1 nodo, 0 Dummy leaf.

Si possono usare due strategie per la numerazione:

- Plain: Assegnare nel bitvector che va da 1 a $2m+1$

- Bold: Numero Rank del nodo che va da 1 a n

- Rank_i: Transf da Plain a Bold

- Select_i: Transf da Bold a Plain

Dato un nodo con Bold label

- Left child = Rank_i(2x)

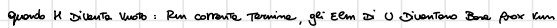
- Right child = Rank_i(2x+1)

- Parent = $\left\lfloor \frac{\text{Select}_i(x)}{2} \right\rfloor$

Un nodo è foglia se ha due Dummy leaves come figlia

KASAI

- l'algoritmo scompone i suffissi in ordine di posizione nel testo.



Lexicographic Search Patricia Tree Front Coded



1) Discesa: Scendo finché ho match sul carattere

- Se arrivo ad una foglia la stringa esiste
- Se non esiste mi fermo a quel nodo e scelgo una foglia qualsiasi del sottoalbero

2) Colloca LCP fra le stringhe che cerco e la stringa considerata nel sottoalbero $\leadsto$ l

il punto di mismatch è $l+1 \rightarrow$ se $l+1$ è un nodo lo indico



se $l+1$ è su un arco indico il punto sull'arco

3) Trovato la Posizione Tex $P\&$ scelgo sempre il blocco a sx $\leadsto$ far come sono composti; Black ci sta che "qualcosa" sia caduto nel blocco precedente

$P\#$ scelgo il blocco a dx

N.B: Nel blocco